HACKATHON PROJECT REPORT

PUBLIC TRANSPORT COMPLAINT PORTAL

A MERN Stack-Based Smart Complaint Management System

Project Domain	Backend Web Development / Smart Public Transportation
Technology Stack	MongoDB • Express.js • React.js • Node.js
Submitted by	[Your Name / Team Name]
Institution	[College Name]
Hackathon	[Hackathon Name]
Academic Year	2026–2027

A centralized, secure, and traceable platform for reporting and managing public transport complaints.

TABLE OF CONTENTS

- 1. Abstract
- 2. Introduction
- 3. Problem Statement
- 4. Existing System and Limitations
- 5. Proposed System
- 6. Objectives
- 7. Scope of the Project
- 8. Technology Stack
- 9. System Architecture
- 10. System Modules
- 11. Database Design
- 12. REST API Design
- 13. Complaint Lifecycle
- 14. Authentication and Security
- 15. Frontend Design
- 16. Suggested Project Structure
- 17. Implementation Methodology
- 18. Testing Strategy
- 19. Expected Outcomes
- 20. Innovation and Unique Selling Points
- 21. Performance and Scalability
- 22. Future Enhancements
- 23. Limitations
- 24. Conclusion
- 25. References

1. ABSTRACT

Public transportation plays a vital role in daily commuting. However, passengers frequently experience problems such as bus delays, overcrowding, poor vehicle maintenance, staff misconduct, route-related issues, and unsatisfactory service. In many cases, passengers lack a centralized and transparent platform to register complaints and monitor their resolution.

The **Public Transport Complaint Portal** is a web-based application designed to simplify and digitize the public transport complaint management process. The system enables passengers to submit complaints related to buses, routes, locations, and transportation services. Each complaint can be assigned a unique tracking ID, allowing users to monitor its status from submission to resolution.

The application uses MongoDB for data management, Express.js and Node.js for backend services, and React.js for the user interface. RESTful APIs manage users, buses, routes, complaints, locations, and complaint statuses. Role-based access distinguishes passengers from administrators and authorized transport officials.

The proposed system aims to improve transparency, reduce manual recordkeeping, support efficient issue tracking, and provide insights into recurring transportation problems. Future extensions may include GPS services, automated complaint classification, and real-time notifications.

Keywords: MERN Stack, Public Transportation, Complaint Management, MongoDB, REST API, Node.js, Express.js, React.js, Authentication, CRUD Operations.

2. INTRODUCTION

Public transportation systems serve thousands of passengers every day. Service quality influences commuter satisfaction, accessibility, safety, and operational efficiency. Traditional complaint mechanisms such as paper forms, telephone calls, and unstructured online feedback can make it difficult to track complaints, identify recurring problems, and ensure timely resolution.

The Public Transport Complaint Portal addresses these challenges through a centralized digital platform. Passengers can submit and track complaints, while authorized officials manage and resolve them. The project demonstrates backend web development through database modeling, API development, request validation, authentication, authorization, CRUD operations, and error handling.

3. PROBLEM STATEMENT

Develop a MERN-based Public Transport Complaint Portal that enables passengers to report transportation-related problems and allows authorized administrators to manage, track, update, and resolve complaints efficiently. The system must manage bus details, route information, complaint records, locations, users, and complaint statuses using MongoDB. React.js provides the frontend, while Node.js and Express.js implement RESTful APIs. The application must support CRUD operations, secure authentication, role-based authorization, complaint tracking, and administrative management.

4. EXISTING SYSTEM AND ITS LIMITATIONS

- **Lack of centralized records:** Complaints may be stored across separate systems or manual registers.
- **Limited tracking:** Passengers may not know whether a complaint has been received, assigned, or resolved.
- **Delayed action:** Complaints can remain unattended without a structured workflow.
- **Poor accountability:** Responsibility for handling an issue may be unclear.
- **Repeated complaints:** Similar issues affecting the same bus or route may be difficult to identify.
- **Limited reporting:** Authorities may lack consolidated information about categories, locations, and resolution times.

5. PROPOSED SYSTEM

The proposed system provides a centralized web application for submitting, tracking, and managing public transport complaints. Passengers can register or log in, select a bus or route, describe a problem, provide the location, and submit a complaint. The backend validates the information and stores the complaint in MongoDB.

A unique tracking ID is generated for each complaint. Administrators can review complaints, assign them to responsible personnel, update statuses, maintain transport records, and monitor statistics. A modular architecture allows future features to be added without redesigning the entire application.

5.1 Key Features

- Passenger registration and secure login.
- Complaint submission with category, description, bus, route, and location.
- Unique complaint tracking ID and status history.
- Bus and route management.
- Administrator dashboard and assignment of complaints.
- Search, filtering, pagination, and complaint statistics.
- Input validation and consistent error responses.
- Optional image attachments with file validation and secure storage.

6. OBJECTIVES

- Demonstrate practical implementation of the MERN stack.
- Develop RESTful APIs using Node.js and Express.js.
- Implement CRUD operations using MongoDB.
- Design database records for users, buses, routes, complaints, and locations.
- Implement authentication and role-based authorization.
- Provide transparent complaint tracking and a traceable lifecycle.
- Create an administrative workflow for assignment and resolution.
- Validate incoming requests and handle errors consistently.
- Support efficient searching, filtering, and pagination.
- Demonstrate the application through functional testing and a working prototype.

7. SCOPE OF THE PROJECT

Included in the initial scope

- Passenger accounts and authentication.
- Bus and route records.
- Complaint creation, retrieval, and authorized updates.
- Location and complaint category management.
- Complaint status history and assignment.
- Administrator dashboard and basic analytics.

Outside the initial scope

- Direct integration with government transport databases.
- Guaranteed real-time GPS tracking without an appropriate data source.

- Automatic integration with official complaint systems.
- AI-powered classification and external notifications.

These features may be considered later if suitable services, permissions, and data are available.

8. TECHNOLOGY STACK

Technology	Purpose
MongoDB	Stores users, buses, routes, complaints, and status history.
Mongoose	Defines schemas, validation rules, references, and database operations.
Node.js	Executes backend JavaScript.
Express.js	Builds REST APIs and middleware.
React.js	Develops the user interface.
JavaScript	Implements application logic.
JWT / secure sessions	Supports authenticated access.
bcrypt	Hashes passwords securely.
Zod or express-validator	Validates API requests.
dotenv	Manages environment configuration.
GitHub	Version control and collaboration.
Postman	API development and testing.

9. SYSTEM ARCHITECTURE

9.1 Presentation Layer

React.js provides the passenger and administrator interfaces, sends HTTP requests, and displays API responses.

9.2 Application Layer

Node.js and Express.js handle requests, authentication, authorization, business rules, validation, and error handling.

9.3 Data Layer

MongoDB stores application records. Mongoose provides schemas, document validation, references, and database operations.

9.4 Request Flow

- A passenger submits a complaint through React.
- React sends a POST request to the Express API.
- Middleware authenticates the user and validates the request.
- The controller applies business rules and creates the complaint.
- Mongoose saves the record in MongoDB.
- Express returns a structured response with the tracking ID and initial status.
- React displays confirmation and tracking information.

10. SYSTEM MODULES

10.1 User Management Module

Handles accounts, login, profiles, and roles.

- Register passengers and authenticate users.
- Store password hashes rather than plaintext passwords.
- Restrict protected endpoints based on role and permission.

10.2 Bus Management Module

Stores vehicle information.

- Bus number or fleet identifier, vehicle type, operational status, and associated route references.
- Authorized administrators can add, view, update, and deactivate bus records.

10.3 Route Management Module

Maintains route information.

- Route number, name, starting point, destination, ordered stops, and operational status.
- Routes are linked to buses and complaints for route-specific analysis.

10.4 Complaint Management Module

The core module of the application.

- Categories include delays, overcrowding, maintenance, staff behaviour, route or stop issues, cleanliness, safety, and other concerns.
- Records include description, category, location, submission time, status, and complainant.
- Optional fields include bus, route, assigned official, and supporting evidence.

10.5 Location Management Module

Identifies where a transportation problem occurred.

- Supports locality, bus stop, landmark, and optional latitude/longitude.
- Coordinates must be validated and location data protected.

10.6 Complaint Tracking Module

Provides a clear view of complaint progress.

- Displays tracking ID, current status, assigned official, timestamps, and resolution remarks.
- Passengers can access their own records unless a separately authorized public tracking method is implemented.

10.7 Administrator Dashboard

Provides a consolidated view of system activity.

- Total, open, in-progress, and resolved complaints.
- Category-wise, bus-wise, and route-wise complaint summaries.
- Average resolution time calculated from actual records.

11. DATABASE DESIGN

MongoDB supports flexible complaint records, nested status history, and evolving application requirements.

11.1 Users Collection

Field	Type	Description
_id	ObjectId	Unique user identifier.
name	String	User's name.
email	String	Unique login email.
passwordHash	String	Securely hashed password.
role	String	Passenger, admin, or official.
createdAt	Date	Account creation timestamp.

11.2 Buses Collection

Field	Type	Description
_id	ObjectId	Unique bus identifier.
busNumber	String	Bus or fleet number.
vehicleType	String	Vehicle type.
routeIds	Array	Associated route references.
status	String	Active or inactive.
createdAt	Date	Record creation time.

11.3 Routes Collection

Field	Type	Description
_id	ObjectId	Unique route identifier.
routeNumber	String	Route number.
routeName	String	Route display name.
startPoint	String	Starting location.
endPoint	String	Destination.
stops	Array	Ordered list of stops.
status	String	Operational status.

11.4 Complaints Collection

Field	Type	Description
_id	ObjectId	MongoDB document ID.
trackingId	String	Unique complaint reference.
userId	ObjectId	Reference to complainant.
busId / routeId	ObjectId	Related bus or route, if applicable.
category	String	Complaint category.
description	String	Complaint details.
location	Object	Location information.
status	String	Current complaint status.
assignedTo	ObjectId	Assigned official, if applicable.
statusHistory	Array	Previous status changes and audit details.
resolutionRemarks	String	Resolution explanation.
createdAt / updatedAt	Date	Submission and modification times.
resolvedAt	Date	Resolution timestamp, if resolved.

11.5 Relationships

- One user can submit multiple complaints.
- One bus or route can be associated with multiple complaints.
- A complaint belongs to one complainant and may be assigned to an authorized official.
- One complaint can contain multiple status-history entries.

11.6 Recommended Indexes

- Unique user email and unique complaint tracking ID.
- Complaint userId with creation date.
- Complaint status with creation date.
- Complaint routeId and busId where frequently queried.

12. REST API DESIGN

The backend exposes versioned endpoints under `/api/v1`.

12.1 Authentication APIs

Method	Endpoint	Purpose
POST	/api/v1/auth/register	Register a passenger.
POST	/api/v1/auth/login	Authenticate a user.
GET	/api/v1/auth/me	Retrieve authenticated profile.

12.2 Complaint APIs

Method	Endpoint	Purpose
POST	/api/v1/complaints	Create a complaint.
GET	/api/v1/complaints	List authorized complaints.
GET	/api/v1/complaints/:id	Retrieve a complaint.
PATCH	/api/v1/complaints/:id	Update permitted fields.
PATCH	/api/v1/complaints/:id/status	Authorized status change.
GET	/api/v1/complaints/track/:trackingId	Track with access control.

12.3 Bus APIs

Method	Endpoint	Purpose
POST	/api/v1/buses	Add a bus.
GET	/api/v1/buses	List buses.
GET	/api/v1/buses/:id	Retrieve bus details.
PATCH	/api/v1/buses/:id	Update bus information.
DELETE	/api/v1/buses/:id	Delete or deactivate a bus.

12.4 Route APIs

Method	Endpoint	Purpose
POST	/api/v1/routes	Add a route.
GET	/api/v1/routes	List routes.
GET	/api/v1/routes/:id	Retrieve route details.
PATCH	/api/v1/routes/:id	Update route information.
DELETE	/api/v1/routes/:id	Delete or deactivate a route.

12.5 Administration and Analytics APIs

Method	Endpoint	Purpose
PATCH	/api/v1/complaints/:id/assign	Assign an authorized official.
GET	/api/v1/admin/dashboard	Retrieve dashboard statistics.
GET	/api/v1/admin/reports	Retrieve complaint summaries.

Complaint lists should support filters such as status, category, routeId, busId, page, and limit. All endpoints must enforce permissions. Passengers must not be able to update administrative fields or mark their own complaints resolved.

13. COMPLAINT LIFECYCLE

Step 1 — Submission: The passenger chooses a category, enters a description, and supplies available bus, route, and location details.

Step 2 — Validation: The backend validates required fields and verifies referenced records.

Step 3 — Registration: The system creates a complaint, generates a tracking ID, and sets the initial status to SUBMITTED.

Step 4 — Review: An authorized official reviews the complaint and may assign it to responsible staff.

Step 5 — Investigation: The complaint progresses to IN_PROGRESS when work begins; remarks may be recorded.

Step 6 — Resolution: An authorized official marks the complaint RESOLVED and records resolution remarks.

Step 7 — Closure: If required, a separate confirmation or closure stage may be introduced.

Suggested statuses: SUBMITTED, UNDER_REVIEW, IN_PROGRESS, RESOLVED, REJECTED.

Status transitions must be validated on the server. Each successful change must be recorded in history. Reopening can be supported if required by the operational rules.

14. AUTHENTICATION AND SECURITY

- **Password hashing:** Use bcrypt; never store plaintext passwords.
- **Authentication:** Verify credentials and use appropriately configured access tokens or secure sessions.
- **Role-based authorization:** Restrict administrative operations to authorized roles.
- **Ownership validation:** Ensure passengers can access only their own complaint records.
- **Request validation:** Check IDs, field types, string lengths, categories, statuses, and references.
- **Mass-assignment protection:** Accept only explicitly permitted fields.
- **Environment variables:** Keep database credentials and token secrets out of source code.
- **Error handling:** Avoid exposing stack traces and internal details to clients.
- **Rate limiting:** Protect login and sensitive endpoints from excessive requests.
- **HTTPS and CORS:** Use HTTPS and allow only approved frontend origins.
- **Audit history:** Record who changed statuses and when.
- **File-upload protection:** Restrict file types and sizes and use secure storage.

Authentication alone is insufficient. Every protected request must also be checked for permission and record ownership.

15. FRONTEND DESIGN

15.1 Passenger Interface

- Home page and project information.
- Registration and login pages.
- Complaint submission form.
- My Complaints page.
- Complaint tracking page.
- Complaint details and status history.

15.2 Administrator Interface

- Dashboard with summary statistics.
- Complaint management table.
- Search and filter controls.
- Assignment and status update forms.
- Bus and route management pages.

- Reports and complaint-category summaries.

The interface should include loading states, form validation, empty states, success messages, and clear error messages. The layout should be responsive across mobile, tablet, and desktop screens.

16. SUGGESTED PROJECT STRUCTURE

```
public-transport-complaint-portal/
■■■ client/
■ ■■■ src/
■ ■■■ components/
■ ■■■ pages/
■ ■■■ services/
■ ■■■ hooks/
■ ■■■ context/
■ ■■■ App.jsx
■ ■■■ main.jsx
■■■ server/
■ ■■■ config/db.js
■ ■■■ controllers/
■ ■■■ middleware/
■ ■■■ models/
■ ■■■ routes/
■ ■■■ services/
■ ■■■ validators/
■ ■■■ utils/
■ ■■■ app.js
■ ■■■ server.js
■■■ .gitignore
■■■ README.md
■■■ package.json
```

The frontend handles presentation and user interaction. The backend contains models, routes, controllers, middleware, and business logic. A service layer can hold reusable operations such as tracking-ID generation, assignment, and status-transition validation.

17. IMPLEMENTATION METHODOLOGY

Phase 1 — Requirement analysis: Identify user roles, complaint categories, data requirements, endpoints, and lifecycle.

Phase 2 — Database design: Create Mongoose schemas, validation rules, references, timestamps, and indexes.

Phase 3 — Backend development: Configure Express, connect MongoDB, implement CRUD APIs, and add centralized error handling.

Phase 4 — Authentication and authorization: Implement registration, login, password hashing, authentication, and role checks.

Phase 5 — Complaint workflow: Implement tracking IDs, assignment, status transitions, and status history.

Phase 6 — Frontend development: Build passenger and administrator pages and connect them to REST APIs.

Phase 7 — Testing: Test normal operations, invalid inputs, access restrictions, database errors, and lifecycle transitions.

Phase 8 — Deployment and demonstration: Configure environment variables, deploy frontend and backend, connect the production database, and prepare a live demo.

18. TESTING STRATEGY

Test Case	Expected Result
Register with valid details	Account created successfully.
Register with existing email	Validation error returned.
Login with correct credentials	Authentication succeeds.
Login with incorrect credentials	Access denied.
Submit a valid complaint	Complaint saved with tracking ID.
Submit incomplete complaint	Request rejected with validation details.
Retrieve an authorized complaint	Complaint details returned.
Access another user's complaint	Access denied.
Passenger attempts admin operation	Access denied.
Official updates complaint status	Status and history updated.
Invalid status transition	Request rejected.
Filter complaints by status	Matching records returned.
Unknown complaint ID	Appropriate not-found response.
Paginated complaint list	Correct page of records returned.
Database connection failure	Failure handled safely.

Tools may include Postman for API testing and Jest or Vitest with Supertest for automated backend tests. Record actual results after implementing and executing the endpoints.

19. EXPECTED OUTCOMES

- A centralized digital complaint repository.
- Faster complaint registration and retrieval.
- A traceable complaint lifecycle.
- Better visibility into unresolved transportation issues.
- Clear separation of passenger and administrative permissions.
- Efficient management of bus and route records.
- More organized complaint reporting.
- A reusable backend architecture for future extensions.

These are expected benefits of the proposed design. Actual effectiveness should be evaluated through implementation and testing.

20. INNOVATION AND UNIQUE SELLING POINTS

20.1 Smart Complaint Tracking

Each complaint receives a unique tracking ID and chronological status history, allowing passengers and officials to understand progress.

20.2 Route-Wise and Bus-Wise Issue Analysis

The dashboard can aggregate complaints by route, bus, category, and location. Repeated maintenance reports can signal a need for inspection, but do not automatically prove a defect.

20.3 Transparent Resolution History

Each status change records a timestamp, the responsible authorized user, and relevant remarks.

20.4 Complaint Prioritization

Transparent rules can prioritize safety-related categories or repeated reports. Multiple complaints should not automatically be treated as verified incidents.

20.5 Performance Analytics

The system can calculate resolution time, pending counts, category distributions, and route-wise complaint totals from actual database records.

Together, these features make the application a complaint-management and operational-insight platform rather than just a feedback form.

21. PERFORMANCE AND SCALABILITY

- Create indexes for frequently queried fields.
- Use pagination instead of retrieving every complaint at once.
- Apply filtering and sorting on the server.
- Use reusable middleware and service functions.
- Use appropriate MongoDB connection pooling.
- Maintain consistent error handling and logging.
- Apply request and upload size limits.
- Introduce caching for read-heavy operations only when justified by measurements.

Performance improvements should be guided by testing and observed query patterns.

22. FUTURE ENHANCEMENTS

- **GPS-based location:** Interactive map selection with explicit permission when device location is requested.
- **Automated notifications:** Email or other permitted notifications for submission, assignment, and resolution.
- **AI-assisted classification:** Suggest categories from descriptions, with human review for important decisions.
- **Duplicate detection:** Identify potentially similar reports for review rather than automatically discarding them.
- **Multilingual interface:** Support English, Telugu, and other relevant languages.
- **SLA monitoring:** Define expected handling times and flag overdue complaints.
- **Advanced analytics:** Display trends, recurring locations, and resolution performance.
- **Authorized integrations:** Connect to transport data sources when APIs and permissions are available.
- **Progressive Web App:** Provide an app-like mobile experience.

23. LIMITATIONS

The initial implementation depends on the accuracy of passenger-submitted information and administrator-maintained records. The system cannot independently verify every complaint, guarantee that an issue will be resolved, or provide live bus positions without a suitable data source. Its effectiveness depends on authorized staff reviewing and updating records. External notifications, GPS integration, and automated classification require additional services and implementation effort.

24. CONCLUSION

The Public Transport Complaint Portal proposes a structured and transparent approach to managing public transportation complaints. By combining MongoDB, Express.js, React.js, and Node.js, the application demonstrates how a modern web stack can address a practical civic problem.

The project covers key backend web development concepts, including database modeling, CRUD operations, RESTful API design, authentication, authorization, validation, and structured error handling. Its complaint lifecycle, tracking IDs, status history, and administrative analytics provide functionality beyond basic data entry.

The modular design supports future enhancements such as location mapping, notifications, multilingual access, and intelligent complaint analysis.

> ***The central goal is to make public transport complaint handling more accessible, traceable, and accountable through a secure and maintainable digital system.***

25. REFERENCES

- MongoDB Documentation — https://www.mongodb.com/docs/
- Express.js Documentation — https://expressjs.com/
- React Documentation — https://react.dev/
- Node.js Documentation — https://nodejs.org/docs/
- Mongoose Documentation — https://mongoosejs.com/docs/
- JSON Web Token Introduction — https://jwt.io/introduction
- OWASP Cheat Sheet Series — https://cheatsheetseries.owasp.org/

Submission note: Replace the title-page placeholders and update implementation details, screenshots, deployment information, and test results to reflect the actual working application. Do not claim features have been completed or tested until verified.